

INFORME DE MÓDULOS DEL BACKEND

Sistema de Gestión para Restaurante – Sabor Gestión

Documentación Técnica Exhaustiva y Desacoplada de Servicios Backend

PROYECTO

Sabor Gestión (Backend Personal)

AUTOR / DESARROLLADOR

Fernando Banda Téllez

RAMA GIT / ESTADO

main (v1.0.0 — Estable tras auditorías)

TECNOLOGÍAS NÚCLEO

Node.js, Express, TypeScript

BASE DE DATOS Y TIEMPO REAL

MongoDB Atlas, Mongoose, Socket.IO

FECHA DEL DOCUMENTO

Octubre 2026

ALCANCE ESTRICTO: SERVICIOS BACKEND EXCLUSIVAMENTE

El presente documento describe única y exclusivamente la arquitectura, módulos, modelos de datos, endpoints HTTP, eventos WebSocket y políticas de seguridad implementadas en el servidor backend (Node.js/Express/TypeScript). El sistema funciona de manera autónoma como un núcleo transaccional desacoplado de cualquier interfaz gráfica o cliente frontend.

Este informe sustituye cualquier especificación preliminar previa e incorpora los resultados formales de las fases de auditoría funcional y hardening de seguridad (BUG-01 a BUG-05).

ÍNDICE. TABLA DE CONTENIDOS GENERAL

1. INTRODUCCIÓN Y CONTEXTO DEL SISTEMA	Contexto operativo del backend monolítico modular	3
2. OBJETIVO DEL INFORME	Propósito técnico y académico del documento	3
3. ALCANCE DEL SISTEMA BACKEND	Delimitación funcional e inclusiones formales	4
4. ARQUITECTURA EN CAPAS DEL BACKEND	Rutas/Middlewares, Controladores, Servicios y Repositorios	5
5. ROLES DEL SISTEMA Y CONTROL DE ACCESO (RBAC)	Políticas de seguridad transversal para 4 roles	6
6. MÓDULOS FUNCIONALES DEL BACKEND (M01 AL M10)	Especificación detallada de los 10 módulos de negocio	7 - 11
• M01: Autenticación y Control de Acceso	Tokens JWT (8h), bcrypt y verificación stateless	7
• M02: Administración de Empleados y Seguridad	CRUD de usuarios, soft delete, arqueo automático	7
• M03: Gestión de Categorías y Catálogo del Menú	Categorías, platos, imágenes Cloudinary y flag disponible	8
• M04: Disponibilidad Culinaria e Ingredientes	Recetas, ingredientes, motor booleano y notas semánticas	8
• M05: Gestión de Ubicaciones y Mesas	Ubicaciones, mesas, unicidad, estados y socket	9
• M06: Gestión de Pedidos y Comandas	mesas:updated Correlativo PED-XXXX, precios congelados y reapertura	9
• M07: Preparación y Seguimiento Operativo (Cocina)	Máquina de estados finita, transiciones y alertas sonoras	10
• M08: Gestión de Pagos y Cierre de Cuentas	Cálculo autoritativo de subtotal, liquidación y comprobante	10
• M09: Comprobantes Digitales y Pagos QR	Generación QR, simulación académica y envío por Nodemailer	11
• M10: Cierre de Caja y Dashboard Gerencial	Auditoría de caja (CierreCaja) y 8 KPIs en tiempo real	11
7. MATRIZ GENERAL DE MÓDULOS DEL BACKEND	Resumen consolidado de modelos, rutas, eventos y roles	12
8. FLUJO OPERATIVO GENERAL DEL BACKEND	Diagrama secuencial de ciclo de vida de la comanda	13
9. INTEGRACIONES EXTERNAS DEL BACKEND	MongoDB Atlas, Cloudinary, Nodemailer y Socket.IO	14
10. SEGURIDAD TRANSVERSAL DEL BACKEND	JWT, RBAC, prevención de inyecciones y autoridad monetaria	14
11. FUNCIONALIDADES NO IMPLEMENTADAS / FUERA DE ALCANCE	Reservas deprecadas, comensales y pasarelas bancarias	15
12. CONCLUSIONES Y ESTADO DEL BACKEND	Madurez del sistema tras fases de auditoría y hardening	15

1. INTRODUCCIÓN Y CONTEXTO DEL SISTEMA

El proyecto "Sabor Gestión" es una plataforma tecnológica integral diseñada para optimizar los flujos operativos, culinarios y financieros de establecimientos gastronómicos. En su estado de madurez actual, el repositorio comprende exclusivamente el motor de servidor backend, desarrollado sobre el entorno de ejecución Node.js utilizando el framework Express y el lenguaje fuertemente tipado TypeScript. La base de datos persistente está soportada por el servicio administrado MongoDB Atlas a través del Object Data Modeling (ODM) Mongoose.

Una de las características cardinales de este backend es su autonomía y desacoplamiento estructural: no depende de ninguna interfaz de usuario, vista HTML renderizada del lado del servidor ni componentes frontend. El sistema expone una Interfaz de Programación de Aplicaciones RESTful (API REST) rigurosamente estructurada junto con un canal bidireccional en tiempo real basado en WebSockets (Socket.IO). Esto permite que clientes externos heterogéneos —tales como aplicaciones de punto de venta (POS), paneles de administración web o terminales táctiles de meseros y cocineros— interactúen con el servidor de forma segura, estandarizada y reactiva.

El backend actúa como la única entidad con autoridad sobre las reglas de negocio, la integridad referencial de los datos, el ciclo de vida de los pedidos, la disponibilidad de platos y la liquidación financiera de las comandas. Toda lógica de control, validación de tipos, autorización de usuarios y cálculos monetarios se ejecuta de forma centralizada en el servidor, blindando al sistema frente a inconsistencias o ataques derivados de clientes deshonestos.

2. OBJETIVO DEL INFORME

El objetivo central del presente documento es proveer una especificación formal, técnica, exhaustiva y académicamente fundamentada de los módulos funcionales que componen actualmente el backend de "Sabor Gestión".

Específicamente, este informe persigue los siguientes propósitos técnicos:

- **Documentar la Arquitectura en Capas:** Presentar la estructura desacoplada en cuatro capas que rige la organización interna del código fuente en Node.js y TypeScript (Rutas, Controladores, Servicios y Repositorios).
- **Definir los 10 Módulos de Negocio:** Desglosar cada uno de los módulos funcionales existentes (M01 al M10), detallando su propósito, modelos de datos asociados, endpoints HTTP expuestos, roles autorizados y eventos en tiempo real.
- **Formalizar el Flujo Operativo:** Explicar de manera secuencial la interacción entre entidades desde la asignación de mesas hasta el arqueo financiero de caja.
- **Registrar el Hardening de Seguridad:** Incorporar los hallazgos y correcciones definitivas aplicadas durante las fases de auditoría de incidentes (BUG-01 al BUG-05), garantizando que la documentación coincida con el código real.
- **Delimitar Funcionalidades Fuera de Alcance:** Esclarecer con precisión qué módulos o esquemas históricos (como reservas de mesas o stock cuantitativo) han sido removidos o nunca formaron parte del alcance del servidor.

3. ALCANCE DEL SISTEMA BACKEND

El alcance del backend comprende todas las operaciones requeridas para administrar integralmente un restaurante en su operativa diaria dentro del local (atención en mesa). El sistema abarca diez dominios funcionales claramente delimitados:

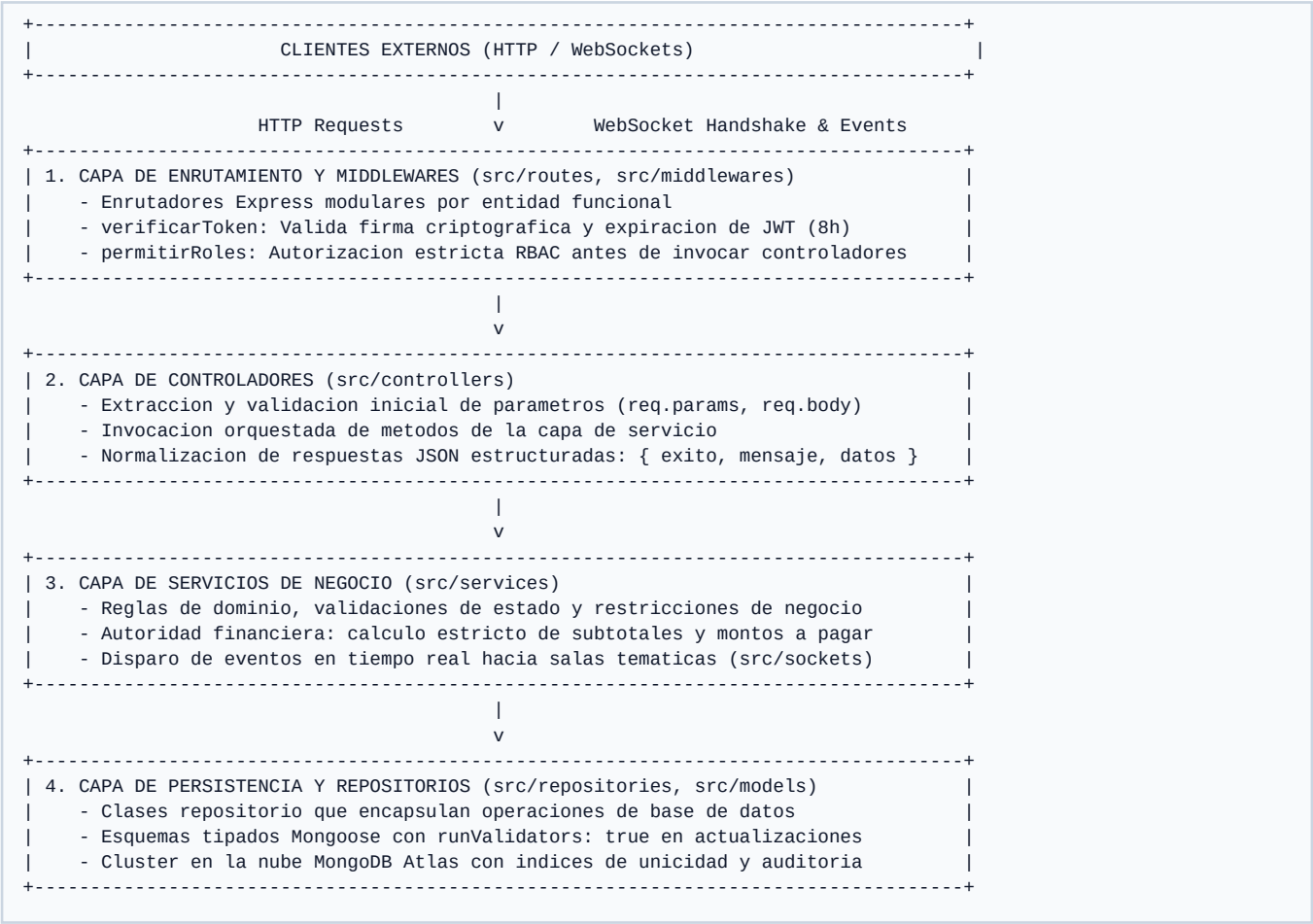
- **Seguridad y Empleados:** Gestión de credenciales, autenticación stateless mediante JWT, control de acceso basado en roles (RBAC) y administración del ciclo de vida del personal con protección de último cajero activo.
- **Catálogo Gastronómico:** Clasificación de platos por categorías, almacenamiento seguro de fotografías en Cloudinary y control de activación o desactivación de platos en carta.
- **Disponibilidad Culinaria e Ingredientes:** Relación entre platos e insumos requeridos mediante recetas, con validación de disponibilidad booleana y evaluación semántica de notas especiales de comanda (ej. omitir ingredientes excluidos).
- **Infraestructura de Salón:** Configuración de sectores físicos (ubicaciones) y mesas, con restricción de unicidad de numeración por sector y control de estados en tiempo real.
- **Comandas y Pedidos:** Emisión correlativa atómica de órdenes (PED-XXXX), fijación inmutable de precios de catálogo al ordenar y adición controlada de nuevos productos a órdenes activas.
- **Operativa de Cocina:** Monitoreo de órdenes en preparación, validación estricta de transiciones de estado de cocina y despacho de alertas sonoras en tiempo real hacia los meseros.
- **Liquidación Financiera:** Cálculo autoritativo en backend de bases imponibles, descuentos comerciales, impuestos y propinas voluntarias, culminando en la liberación automática de mesas.
- **Comprobantes y Pagos QR:** Emisión digital de tickets de pago, envío automatizado de recibos formateados por correo electrónico y soporte para webhooks simulados de transferencias QR.
- **Arqueo de Caja y Analítica Gerencial:** Registro estructurado de cierres de turno de cajeros y generación de dashboards con ocho indicadores clave de rendimiento (KPIs) calculados en tiempo real mediante pipelines de agregación.

DELIMITACIÓN DE EXCLUSIONES DEL SERVIDOR

El alcance actual del sistema NO contempla clientes comensales directos (usuarios externos), pedidos a domicilio (delivery), pasarelas de pago bancarias reales (el flujo QR es una simulación académica deliberada) ni reservas anticipadas de mesas (las cuales fueron retiradas deliberadamente en el commit a1edce2).

4. ARQUITECTURA EN CAPAS DEL BACKEND

El backend implementa un patrón arquitectónico en cuatro capas concéntricas fuertemente desacopladas. Esta estructura garantiza la separación de responsabilidades, la alta cohesión interna, la facilidad para efectuar pruebas automatizadas y la capacidad de evolucionar componentes sin generar efectos colaterales indeseados.



Descripción de las 4 Capas Arquitectónicas:

- 1. Rutas y Middlewares:** Intercepta peticiones HTTP. verificarToken decodifica el JWT e inyecta la identidad en req.usuario. permitirRoles aborta con 403 Forbidden si el rol no está autorizado.
- 2. Controladores:** Recibe requests, valida ObjectIds de MongoDB y estandariza las respuestas en contratos homogéneos: { exito: true/false, mensaje, datos/error }.
- 3. Servicios de Negocio:** Cerebro transaccional del sistema: valida disponibilidad culinaria, gestiona máquinas de estado de pedidos y calcula montos oficiales sin confiar en inputs del cliente.
- 4. Persistencia y Repositorios:** Encapsula operaciones Mongoose sobre MongoDB Atlas, forzando runValidators: true en updates para garantizar la integridad de enums y esquemas.

5. ROLES DEL SISTEMA Y CONTROL DE ACCESO (RBAC)

El backend implementa un modelo de Control de Acceso Basado en Roles (RBAC — Role-Based Access Control) fundamentado en el enum tipado "RolUsuario" definido en el modelo Usuario. Cada usuario activo del sistema posee exactamente un rol asignado, el cual determina el subconjunto de endpoints y acciones que tiene permitido invocar.

Rol del Sistema	Dominio Principal	Responsabilidades y Endpoints Permitidos
Administrador	Gobierno Global y Configuración	Acceso irrestricto a todos los endpoints del backend. Gestión de empleados (CRUD usuarios), configuración de ubicaciones y mesas, catálogo de categorías y platos, recetas e inventario de ingredientes, anulación de pedidos, cobros extraordinarios y visualización del Dashboard gerencial.
Mesero	Atención en Piso y Comandas	Gestión de mesas en salón (marcar ocupada, cambiar de estado), creación de nuevos pedidos (POST /api/pedidos), adición de ítems a pedidos existentes (PUT /api/pedidos/:id), cancelación de pedidos abiertos y consulta del catálogo e ingredientes disponibles.
Cocinero	Despacho y Cocina	Monitoreo del listado de pedidos activos, transición operativa de estados de cocina (PATCH /api/pedidos/:id/estado hacia EN_PREPARACION y ENTREGADO), y visualización técnica de recetas e ingredientes para preparación.
Cajero	Operativa Financiera y Caja	Consulta de pedidos pendientes de cobro, procesamiento final de pagos (POST /api/pagos/:pedidoId/procesar), generación y simulación de pagos QR, despacho de recibos digitales por correo electrónico y registro de arqueos de caja al finalizar turno.

POLÍTICA DE PRIVILEGIO MÍNIMO Y RECHAZO DETERMINISTA

Cualquier intento de invocar un endpoint con un rol no autorizado es rechazado de forma determinista por el middleware "permitirRoles" en la capa HTTP con código 403 Forbidden antes de alcanzar la lógica de negocio, impidiendo escalamiento de privilegios.

6. MÓDULOS FUNCIONALES DEL BACKEND (M01 AL M10)

A continuación se detallan exhaustivamente los diez módulos funcionales del backend actual, documentando modelos Mongoose, endpoints, roles autorizados, eventos WebSocket y reglas de negocio críticas.

M01 — Autenticación y Control de Acceso

MODELOS MONGOOSE: Usuario (src/models/Usuario.ts)

ROLES AUTORIZADOS: Público (Login) / Todos los roles autenticados (Validación)

EVENTOS WEBSOCKET: Ninguno

M01 valida la identidad del personal y emite credenciales criptográficas de sesión. Implementa una arquitectura completamente stateless mediante JSON Web Tokens (JWT), eliminando almacenamiento de sesiones en memoria.

Método	Ruta HTTP	Roles Permitidos	Descripción Funcional
POST	/api/usuarios/login	Público	Valida credenciales (username/password) y retorna token JWT y perfil.

Reglas de Negocio Críticas:

- **Hashing con bcrypt:** Contraseñas procesadas con bcrypt (salt rounds = 10); nunca se guardan en texto plano.
- **Vigencia y Payload JWT:** Tokens firmados con clave secreta simétrica, vigencia de 8h y payload mínimo: { id, rol }.
- **Verificación de Empleado Activo:** Login verifica que "estado === true". Empleados desactivados retornan HTTP 401.

M02 — Administración de Empleados y Seguridad

MODELOS MONGOOSE: Usuario (src/models/Usuario.ts), CierreCaja (src/models/CierreCaja.ts)

ROLES AUTORIZADOS: Administrador

EVENTOS WEBSOCKET: Ninguno

M02 administra el personal del restaurante. Permite registrar empleados, asignarles rol, modificar credenciales y gestionar su ciclo de vida operativo mediante bajas lógicas o físicas.

Método	Ruta HTTP	Roles Permitidos	Descripción Funcional
GET	/api/usuarios	Administrador	Lista empleados registrados con soporte de paginación y filtros.
GET	/api/usuarios/:id	Administrador	Obtiene la información de un empleado específico por su ObjectId.
POST	/api/usuarios	Administrador	Registra un nuevo empleado con password encriptado y rol asignado.
PUT	/api/usuarios/:id	Administrador	Actualiza datos personales, rol o credenciales de un empleado.
DELETE	/api/usuarios/:id	Administrador	Elimina físicamente a un empleado de la base de datos (Hard Delete).
PATCH	/api/usuarios/:id/estado	Administrador	Cambia estado activo/inactivo (Soft Delete) y registra arqueo si aplica.

Reglas de Negocio Críticas:

- **Protección de Último Cajero Activo:** Tanto en DELETE como en PATCH, se aborta con HTTP 400 si se intenta desactivar al único cajero activo.
- **Arqueo Automático al Desactivar Cajero:** Si se desactiva un Cajero con el campo "reporte", se crea automáticamente un documento en CierreCaja.

M03 — Gestión de Categorías y Catálogo del Menú

MODELOS MONGOOSE: Categoría (src/models/Categoría.ts), Plato (src/models/Plato.ts)

ROLES AUTORIZADOS: Administrador (Mutaciones) / Todos los roles (Lectura)

EVENTOS WEBSOCKET: Ninguno

M03 estructura la oferta gastronómica del restaurante. Administra las familias o categorías gastronómicas y la ficha técnica de cada plato, incluyendo nombre, descripción, precio oficial, disponibilidad booleana e imagen en Cloudinary.

Método	Ruta HTTP	Roles	Descripción Funcional
GET	/api/categorias	Todos	Consulta todas las categorías activas registradas en el catálogo.
POST	/api/categorias	Admin	Crea una nueva categoría gastronómica con nombre único.
PUT	/api/categorias/:id	Admin	Actualiza el nombre, descripción o estado de una categoría.
DELETE	/api/categorias/:id	Admin	Elimina una categoría del catálogo del restaurante.
GET	/api/platos	Todos	Lista los platos de la carta (soporta filtro por categoría y estado).
GET	/api/platos/:id	Todos	Obtiene la ficha técnica completa de un plato específico.
POST	/api/platos	Admin	Registra un nuevo plato asignándole precio, categoría e imagen.
PUT	/api/platos/:id	Admin	Actualiza información del plato, incluyendo su disponibilidad booleana.
DELETE	/api/platos/:id	Admin	Elimina un plato del catálogo gastronómico.
POST	/api/upload	Admin	Sube fotografía de plato a Cloudinary mediante Multer en memoria.

Reglas de Negocio Críticas:

- **Disponibilidad Booleana de Platos:** Plato maneja "disponible: boolean" (no existe enum "Agotado"). Se actualiza vía PUT / api/platos/:id.
- **Carga Multipart en Memoria:** Multer utiliza memoryStorage; la imagen se transmite por stream seguro a Cloudinary.

M04 — Disponibilidad Culinaria e Ingredientes

MODELOS MONGOOSE: Ingrediente (src/models/Ingrediente.ts), Receta (src/models/Receta.ts), Plato

ROLES AUTORIZADOS: Administrador, Cocinero (Gestión) / Mesero (Consulta)

EVENTOS WEBSOCKET: Ninguno

M04 evalúa la viabilidad técnica de preparar un plato verificando que los ingredientes de su receta estén disponibles. No es un inventario físico con conteo numérico de porciones, sino un motor de disponibilidad lógica en cocina.

Método	Ruta HTTP	Roles	Descripción Funcional
GET	/api/inventario/ingredientes	Admin, Cocinero	Lista ingredientes con su estado de disponibilidad booleana.
POST	/api/inventario/ingredientes	Admin, Cocinero	Registra un nuevo ingrediente en el catálogo de cocina.
PUT	/api/inventario/ingredientes/:id	Admin, Cocinero	Actualiza nombre, unidad o estado disponible del ingrediente.
DELETE	/api/inventario/ingredientes/:id	Admin	Elimina un ingrediente del catálogo.
GET	/api/inventario/recetas	Admin, Cocinero	Lista recetas vinculando platos con ingredientes requeridos.
POST	/api/inventario/recetas	Admin, Cocinero	Define o actualiza la receta de insumos de un plato.
GET	/api/inventario/estado	Todos	Consulta el estado de platos según disponibilidad de insumos.

Reglas de Negocio Críticas:

- **Motor validarDisponibilidadIngredientes:** Verifica que ningún ingrediente obligatorio esté agotado antes de ordenar.
- **Exclusión Semántica por Observaciones:** Notas como "sin cebolla" omiten el ingrediente de la validación, permitiendo pedir el plato.

M05 — Gestión de Ubicaciones y Mesas

MODELOS MONGOOSE: Ubicacion (src/models/Ubicacion.ts), Mesa (src/models/Mesa.ts)

ROLES AUTORIZADOS: Administrador (CRUD) / Mesero, Administrador (Estado)

EVENTOS WEBSOCKET: mesas:updated

M05 gestiona las áreas físicas (Terraza, Salón, Barra) y las mesas asociadas a cada una, controlando su estado operativo y sincronizándolo en tiempo real con todas las terminales conectadas vía WebSockets.

Método	Ruta HTTP	Roles	Descripción Funcional
GET	/api/ubicaciones	Todos	Obtiene la lista de áreas o zonas físicas del restaurante.
POST	/api/ubicaciones	Admin	Registra una nueva zona física en el establecimiento.
PUT	/api/ubicaciones/:id	Admin	Actualiza los datos descriptivos de una ubicación.
DELETE	/api/ubicaciones/:id	Admin	Elimina una zona si no posee mesas activas asociadas.
GET	/api/mesas	Todos	Consulta todas las mesas registradas con ubicación y estado actual.
POST	/api/mesas	Admin	Crea una nueva mesa asignándole número, capacidad y ubicación.
PUT	/api/mesas/:id	Admin	Actualiza capacidad o reasigna ubicación de la mesa.
DELETE	/api/mesas/:id	Admin	Elimina físicamente una mesa del sistema.
PATCH	/api/mesas/:id/estado	Mesero, Admin	Modifica el estado de la mesa y emite evento WebSocket.

Reglas de Negocio Críticas:

- **Estados en MongoDB:** Enum oficial: "Libre", "Ocupada", "Cuenta Solicitada". Aliases como "Disponible" son normalizados.
- **Validación Estricta (BUG-05):** findByIdAndUpdate con runValidators: true impide persistir estados fuera del enum (HTTP 400).
- **Evento mesas:updated:** Cualquier mutación en PATCH /mesas/:id/estado emite el evento en tiempo real a todas las salas.

M06 — Gestión de Pedidos y Comandas

MODELOS MONGOOSE: Pedido (src/models/Pedido.ts), Contador (src/models/Contador.ts), Mesa, Plato

ROLES AUTORIZADOS: Mesero, Administrador

EVENTOS WEBSOCKET: Ninguno directo (Delega a Cocina en M07)

M06 administra las comandas de los comensales. Genera correlativos atómicos, fija los precios unitarios oficiales del catálogo, calcula importes y permite la adición ordenada de nuevos ítems a pedidos en curso.

Método	Ruta HTTP	Roles	Descripción Funcional
GET	/api/pedidos	Todos	Consulta pedidos con soporte de filtros por estado, mesa y fechas.
GET	/api/pedidos/:id	Todos	Obtiene la comanda completa con el desglose de ítems y totales.
POST	/api/pedidos	Mesero, Admin	Crea una nueva comanda, asigna correlativo PED-XXXX y ocupa la mesa.
PUT	/api/pedidos/:id	Mesero, Admin	Agrega platos adicionales a un pedido existente recalculando subtotales.

Reglas de Negocio Críticas:

- **Correlativo Atómico PED-XXXX:** Generado mediante findOneAndUpdate atómico sobre Contador, previniendo colisiones.
- **Fijación Oficial de Precios:** El backend consulta el precio oficial en Plato e ignora precios suministrados por el cliente.
- **Reapertura Automática:** Si se agregan platos a un pedido en ENTREGADO, el estado se reabre automáticamente a ABIERTO.
- **Protección Mass Assignment (BUG-02):** PUT /pedidos/:id ignora campos financieros (total, subtotal) del body.

M07 — Preparación y Seguimiento Operativo (Cocina)**MODELOS MONGOOSE:** Pedido (src/models/Pedido.ts)**ROLES AUTORIZADOS:** Cocinero, Administrador (Estados) / Mesero, Administrador (Cancelación)**EVENTOS WEBSOCKET:** pedidos:lista

M07 orquesta el flujo de cocina y la comunicación asíncrona entre cocineros y meseros. Controla las transiciones de estado de las comandas y emite alertas en tiempo real con timbre sonoro al finalizar los platos.

Método	Ruta HTTP	Roles	Descripción Funcional
PATCH	/api/pedidos/:id/estado	Cocinero, Admin	Avanza el estado del pedido: ABIERTO -> EN_PREPARACION -> ENTREGADO.
PATCH	/api/pedidos/:id/cancel	Mesero, Admin	Cancela un pedido si no ha sido finalizado ni cobrado.

Reglas de Negocio Críticas:

- **Máquina de Estados Finita (BUG-01):** Transiciones estrictas: ABIERTO -> EN_PREPARACION -> ENTREGADO. Saltos inválidos retornan HTTP 400.
- **Alerta Sonora pedidos:lista:** Al marcar ENTREGADO, emite WebSocket a meseros con audioAlert: true para activar campana sonora.
- **Inmutabilidad de Cierres:** Un pedido en estado CERRADO o CANCELADO no puede ser alterado bajo ninguna circunstancia.

M08 — Gestión de Pagos y Cierre de Cuentas**MODELOS MONGOOSE:** Pedido (src/models/Pedido.ts), Pago (src/models/Pago.ts), Mesa (src/models/Mesa.ts)**ROLES AUTORIZADOS:** Cajero, Administrador**EVENTOS WEBSOCKET:** mesas:updated

M08 es el núcleo transaccional financiero. Liquidada las cuentas, aplica descuentos comerciales válidos, calcula impuestos (IVA), procesa propinas, asienta el pago, cierra la comanda y libera automáticamente la mesa.

Método	Ruta HTTP	Roles	Descripción Funcional
GET	/api/pedidos/pendientes-cobro	Cajero, Admin	Lista todos los pedidos con cuenta solicitada pendientes de cobro.
POST	/api/pagos/:pedidoid/procesar	Cajero, Admin	Procesa cobro final, registra pago, cierra comanda y libera mesa.

Reglas de Negocio Críticas y Autoridad Monetaria:

- **Autoridad Absoluta del Subtotal (BUG-03):** El backend ignora subtotalCierre del body y calcula la base real desde los detalles persistidos.
- **Transacción de Liquidación:** Calcula IVA y descuentos, pasa Pedido a CERRADO, libera Mesa a Libre (mesas:updated) y crea Pago (Pagado).
- **Métodos de Pago Homologados:** Soporta liquidación mediante Efectivo, Tarjeta y Transferencia / QR.

M09 — Comprobantes Digitales y Pagos QR

MODELOS MONGOOSE: Pedido (src/models/Pedido.ts), Pago (src/models/Pago.ts)

ROLES AUTORIZADOS: Cajero, Administrador / Simulación Pública (notificar-qr)

EVENTOS WEBSOCKET: caja:notificacion_qr

M09 complementa la gestión de cobro permitiendo la generación de códigos QR simulados, la recepción de webhooks de notificación y la emisión de comprobantes digitales con envío automatizado por correo vía SMTP.

Método	Ruta HTTP	Roles	Descripción Funcional
POST	/api/pagos/generar-qr/:pedidoId	Cajero, Admin	Genera payload/código QR codificado con los datos del cobro pendiente.
POST	/api/pagos/notificar-qr/:pedidoId	Público (Sim.)	Webhook de simulación QR; emite notificación a cajeros (BUG-04).
POST	/api/pagos/:pedidoId/enviar-recibo	Cajero, Admin	Genera y envía comprobante formal en formato HTML vía Nodemailer.

Reglas de Negocio Críticas:

- **Simulación Académica Segura (BUG-04):** notificar-qr no liquida el pago en BD; emite WebSocket caja:notificacion_qr para cobro formal en caja.
- **Recibo Formal por Nodemailer:** Compila plantilla HTML profesional con datos fiscales, desglose de ítems, propina, impuestos y fecha.

M10 — Cierre de Caja y Dashboard Gerencial

MODELOS MONGOOSE: CierreCaja (src/models/CierreCaja.ts), Pago, Pedido, Mesa, Usuario

ROLES AUTORIZADOS: Administrador

EVENTOS WEBSOCKET: Ninguno

M10 provee auditoría financiera y analítica gerencial en tiempo real. Administra los arqueos de caja al cierre de turno y expone ocho indicadores clave de rendimiento (KPIs) calculados mediante agregaciones en MongoDB Atlas.

Método	Ruta HTTP	Roles	Descripción Funcional
GET	/api/dashboard/resumen	Administrador	Retorna en tiempo real los 8 KPIs gerenciales agregados del restaurante.
POST	/api/dashboard/cierre-caja	Administrador	Registra formalmente el arqueo y balance financiero de un turno de caja.

Los 8 Indicadores Clave de Rendimiento (KPIs) Gerenciales:

- **1. Total de Ventas:** Suma monetaria acumulada de todas las transacciones en estado Pagado del período.
- **2. Total de Pedidos:** Cantidad neta de comandas completadas satisfactoriamente (estado CERRADO).
- **3. Ticket Promedio:** Monto promedio gastado por comanda procesada (Ventas Totales / Pedidos Totales).
- **4. Ventas por Método de Pago:** Distribución agregada del volumen monetario entre Efectivo, Tarjeta y QR.
- **5. Platos Más Vendidos:** Ranking de popularidad gastronómica ordenado por unidades ordenadas.
- **6. Pedidos por Hora:** Distribución temporal de órdenes a lo largo del día para detectar picos de demanda.
- **7. Tasa de Ocupación de Mesas:** Porcentaje de mesas actualmente Ocupada o Cuenta Solicitada respecto al total.
- **8. Rendimiento del Personal:** Productividad individual cuantificada por mesero (pedidos) y cajero (cobros procesados).

7. MATRIZ GENERAL DE MÓDULOS DEL BACKEND

La siguiente tabla consolida los diez módulos funcionales del backend, resumiendo sus componentes arquitectónicos clave:

Cód.	Módulo Funcional	Modelos Mongoose	Endpoints Clave	WebSockets	Roles
M01	Autenticación & Acceso	Usuario	POST /login	Ninguno	Público, Todos
M02	Admin. Empleados	Usuario, CierreCaja	CRUD /usuarios	Ninguno	Admin
M03	Menú y Categorías	Categoría, Plato	CRUD /categorias, /platos	Ninguno	Admin, Todos
M04	Disponibilidad Culinaria	Ingrediente, Receta	/inventario/*	Ninguno	Admin, Cocinero
M05	Ubicaciones y Mesas	Ubicacion, Mesa	CRUD /mesas, PATCH /estado	mesas:updated	Admin, Mesero
M06	Pedidos y Comandas	Pedido, Contador	POST /pedidos, PUT /:id	Ninguno	Admin, Mesero
M07	Cocina y Seguimiento	Pedido	PATCH /estado, /cancel	pedidos:listo	Admin, Cocinero
M08	Pagos y Liquidación	Pedido, Pago, Mesa	POST /pagos/:id/procesar	mesas:updated	Admin, Cajero
M09	Comprobantes y QR	Pedido, Pago	/generar-qr, /enviar-recibo	caja:notif_qr	Admin, Cajero
M10	Cierre Caja & Dashboard	CierreCaja, Pago	GET /dashboard/resumen	Ninguno	Admin

CONSISTENCIA DE CONTRATOS Y TIPOS EN EL BACKEND

Cada uno de los 10 módulos expuestos cuenta con interfaces TypeScript dedicadas (en src/types/), esquemas Mongoose con validación en base de datos y documentación OpenAPI sincronizada en docs/openapi.yaml, asegurando total predictibilidad operativa.

8. FLUJO OPERATIVO GENERAL DEL BACKEND

El siguiente diagrama secuencial ilustra el ciclo de vida completo de una comanda en el restaurante, desde la asignación de la mesa hasta el arqueo final de caja, evidenciando la interacción coordinada entre los diferentes módulos y capas del backend:

PASO 1: ASIGNACION Y OCUPACION DE MESA (Modulo M05)

```
Mesero ejecuta PATCH /api/mesas/:id/estado { "estado": "Ocupada" }  
\--> Backend valida transicion -> Persiste en MongoDB -> Emite "mesas:updated"
```

PASO 2: CONSULTA DE MENU Y VALIDACION CULINARIA (Modulos M03 y M04)

```
Mesero consulta GET /api/platos y GET /api/inventario/estado  
\--> Backend filtra platos segun disponibilidad de insumos en cocina
```

PASO 3: APERTURA Y GENERACION DE COMANDA (Modulo M06)

```
Mesero envia POST /api/pedidos { mesaId, detalles: [{ platoId, cantidad, notas }] }  
\--> Contador atomico genera "PED-XXXX" -> Precios oficiales congelados -> Guarda Pedido
```

PASO 4: PREPARACION Y MONITOREO DE COCINA (Modulo M07)

```
Cocinero avanza estado mediante PATCH /api/pedidos/:id/estado  
|--> 4a: { estado: "EN_PREPARACION" } -> Orden entra a fogones  
\--> 4b: { estado: "ENTREGADO" } -> Backend emite WebSocket "pedidos:listo" (alerta sonora)
```

PASO 5: SOLICITUD DE CUENTA (Modulo M05)

```
Mesero solicita cuenta mediante PATCH /api/mesas/:id/estado { "estado": "Cuenta Solicitada" }  
\--> Mesa pasa a "Cuenta Solicitada" -> Cajero visualiza pedido en pendientes-cobro
```

PASO 6: PROCESAMIENTO DE PAGO Y CIERRE (Modulo M08 y M09)

```
Cajero ejecuta POST /api/pagos/:pedidoId/procesar { metodoPago, propina, descuento }  
|--> Backend ignora subtotal del cliente y calcula la base real oficial  
|--> Crea documento Pago ("Pagado") -> Pedido pasa a "CERRADO"  
\--> Mesa se libera ("Libre") automaticamente -> Emite WebSocket "mesas:updated"
```

PASO 7: COMPROBANTE DIGITAL Y CIERRE DE TURNO (Modulos M09 y M10)

```
--> Cajero envia comprobante digital por email via POST /api/pagos/:id/enviar-recibo  
\--> Administrador audita arqueo en POST /cierre-caja y supervisa GET /dashboard/resumen
```

ORQUESTACIÓN ATÓMICA DE ESTADOS

Nótese cómo el backend actúa como director de orquesta: una acción en el módulo de pagos (M08) actualiza atómicamente la entidad Pedido (M06) y la entidad Mesa (M05), disparando notificaciones en tiempo real vía WebSockets a todas las terminales conectadas sin intervención del usuario.

9. INTEGRACIONES EXTERNAS DEL BACKEND

El backend se comunica con cuatro plataformas y protocolos de infraestructura externa esenciales para su operación:

- **MongoDB Atlas:** Base de datos NoSQL documental en la nube gestionada a través del ODM Mongoose. Provee alta disponibilidad, índices de unicidad en colecciones clave y transacciones atómicas de sesión.
- **Cloudinary API:** Servicio en la nube especializado en el almacenamiento, compresión y distribución CDN de activos visuales gastronómicos. Las imágenes se transfieren mediante Multer en memoria sin escribir en disco.
- **Nodemailer / SMTP:** Motor de correo electrónico para la generación y envío asíncrono de comprobantes de pago digitales con maquetación HTML.
- **Socket.IO (WebSockets):** Servidor de comunicación bidireccional en tiempo real montado sobre el servidor HTTP nativo de Node.js. Soporta salas lógicas (rooms) para meseros, cocineros y cajeros.

10. SEGURIDAD TRANSVERSAL DEL BACKEND

A lo largo de todo el ciclo de peticiones, el backend aplica múltiples barreras de protección de datos, validación semántica y autorización de recursos:

- **Autenticación Stateless JWT:** Toda ruta protegida exige un token firmado por clave simétrica con caducidad estricta (8 horas), evitando vectores de ataque por sesiones persistentes obsoletas.
- **Control RBAC Centralizado:** El middleware `permitirRoles` valida a nivel de enrutador que el usuario tenga privilegios para la operación solicitada antes de invocar la capa de servicio.
- **Protección Criptográfica bcrypt:** Las contraseñas de los usuarios son procesadas con 10 rondas de salt antes de su almacenamiento en la base de datos.
- **Validación Estricta de Esquemas (runValidators):** Todas las mutaciones parciales de repositorio ejecutan validaciones completas contra el esquema Mongoose, previniendo datos inválidos en la base de datos.
- **Autoridad Financiera Absoluta en Backend:** El servidor calcula autónomamente precios, subtotales, impuestos y montos de cobro. Cualquier parámetro financiero suministrado por el cliente es descartado.
- **Manejo Centralizado de Excepciones:** Los controladores capturan errores de forma determinista, evitando fugas de información interna y retornando códigos HTTP semánticos (400, 401, 403, 404, 500).

11. FUNCIONALIDADES NO IMPLEMENTADAS / FUERA DE ALCANCE

Para evitar desalineaciones con clientes o integraciones externas, se documentan explícitamente los módulos y características que NO forman parte del alcance del backend actual:

- **Módulo de Reservas de Mesas:** Fue eliminado deliberadamente en el commit a1edce2 para simplificar la operación del restaurante. No existen rutas /api/reservas ni identificadores RES-XXXX.
- **Portal de Comensales:** El sistema no gestiona cuentas para clientes finales; todos los usuarios del sistema corresponden exclusivamente a personal del restaurante.
- **Pasarelas Bancarias Reales:** El flujo de pagos QR y confirmación vía webhook es una simulación académica local; no existe integración con instituciones financieras bancarias reales.
- **Control Numérico de Stock (Kardex):** El inventario culinario se gestiona mediante flags de disponibilidad booleana; no existe decremento automático de unidades métricas o existencias de bodega.

12. CONCLUSIONES Y ESTADO DEL BACKEND

El backend de "Sabor Gestión" se consolida como una solución robusta, modular, escalable y fuertemente tipada para la administración integral de un establecimiento gastronómico. La rigurosa separación de responsabilidades en cuatro capas garantiza que la lógica de negocio resida exclusivamente en el servidor, manteniendo al sistema blindado e independiente de las interfaces de usuario.

Las auditorías de calidad y los procesos de hardening ejecutados han erradicado vulnerabilidades críticas de negocio (transiciones ilegítimas de comandas, mass assignment financiero, alteración arbitraria de subtotales, cierres automáticos inseguros de pagos QR y corrupción de estados de mesa). Como resultado, el backend se encuentra en un estado de alta estabilidad técnica y completitud funcional, listo para operar como núcleo transaccional en producción o integrarse con nuevos clientes web o móviles.

FERNANDO BANDA TÉLLEZ

Desarrollador y Arquitecto de Software • Sabor Gestión Backend

ESTADO: REPOSITORIO AUDITADO Y APROBADO